

INDIAN INSTITUTE OF TECHNOLOGY MADRAS

Learning from Comparison Data

M.Tech Project: Phase I Report

Under the Supervision of

Prof. Suryanarayana Sankagiri

Submitted by

Nitesh Kumar Shah

ID25M806

Department of Data Science and Artificial Intelligence

Indian Institute of Technology Madras

Chennai – 600036

November 2026

Abstract

Classical choice models such as Bradley terry Luce(BTL) model assumes that the relative preference between two alternatives remains unchanged when other alternatives are added or removed. This property is called Independence of Irrelevant Alternatives (IIA). However, the available alternatives can influence a person's choice, which leads to the violations of IIA. Motivated by Discovering Context Effects from Raw Choice Data by Seshadri et al., this project studies how such context effects can be represented and learned from observed choice data.

This project begins with the Bradley–Terry model to understand parameter recovery and estimation from pairwise comparison data by generating synthetic data, we studied parameter recovery, the effects of comparison data, the existence of a finite maximum likelihood estimates, and the behaviour of different optimization methods. We then studied and implemented the Context-Dependent Model (CDM), which allows an item's utility to depend on the other items in the choice set.

The experiments conducted so far helped us to examine how the CDM recovery changes with the number of observations and items. We also fit Bradley–Terry and CDM to data generated by each other and compare their predictive performance on unseen choices. Finally, we investigated the low-rank CDM as a way to reduce the number of model parameters and study its computational behaviour. The results illustrates that the context-dependent model improves prediction in the synthetically generated choice data.

The code and experiment notebooks are available in the project's GitHub repository.

1 Introduction

1.1 Motivation

Let us try to imagine that someone has to get from IIT Madras to the airport, and they have 4 real choices like Car, Bike, taxi, bus.. And so, based on the cost, comfort, speed they have some natural preference ranking among this 4, let's say they choose or pick

$$\text{Car} > \text{Taxi} > \text{Bike} > \text{Bus}$$

Now, suppose we add a 5th, a clear irrelevant option to the list which would almost take 3 hours and nobody would seriously choose this. Common sense tells us that adding 'walking' doesn't really change the feel about the car v/s the taxi v/s the bike, as it is the irrelevant alternative, and their **relative preference** between all these 4 alternative would not change at all.

So this idea is called independence of irrelevant alternatives and the core behind the classical model of choice.

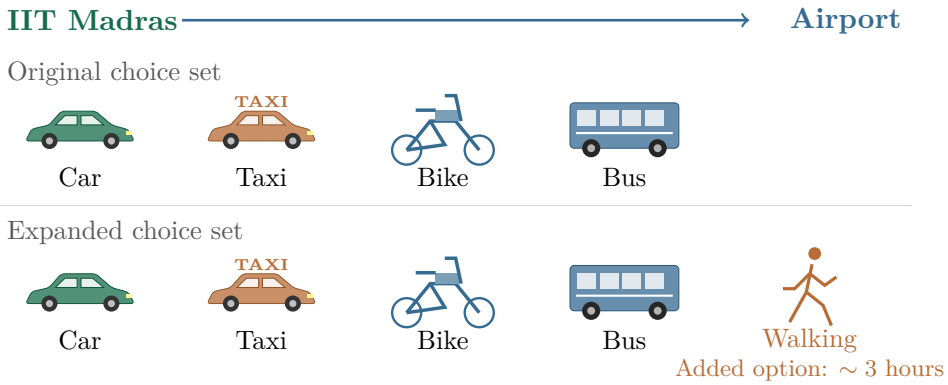


Figure 1: The commute choices from IIT Madras to the airport. Under the IIA assumption, adding walking leaves the relative odds between the original alternatives unchanged. (Here, the three-hour walking time is part of the illustrative example.)

But the core idea is that maybe human brains may not be registering the preference in the same way of that of multinomial logits. As there is a strong experimental evidence proves that it would shift the attractiveness of things in the way that how attractive the existing options really looks → this is actually called violations of independence of irrelevant alternatives.[5]

2 Background and Choice Models

2.1 Choice Data and Choice Sets

Let $\mathcal{X}$ denote the universal set of alternatives. A choice set is a subset of $\mathcal{X}$ containing the alternatives available to a decision-maker at a particular time. As a separate example, consider travel from IIT Madras to the airport. The universal set of transport alternatives is

$$\mathcal{X} = \{\text{Car}, \text{Taxi}, \text{Bike}, \text{Bus}, \text{Walking}\}. \tag{1}$$

The walking alternative remains a member of the universal set even when it does not appear in a particular choice set. All the smaller choice sets are constructed from the same fixed universal set. Let $A, B \subseteq \mathcal{X}$ be two choice sets. Let x and y be alternatives belonging to the intersection $A \cap B$.

2.2 Independence of Irrelevant Alternatives

For a universal set $\mathcal{X}$ and choice sets $A, B \subseteq \mathcal{X}$, the Independence of Irrelevant Alternatives property for two alternatives x and y can be written as

$$\frac{P(x \mid A)}{P(y \mid A)} = \frac{P(x \mid B)}{P(y \mid B)}, \qquad x, y \in A \cap B. \tag{2}$$

The equation states that if x and y are present in both choice sets, then their relative odds should not change merely because other alternatives have been added or removed.

2.3 Multinomial Logit Model

The classical multinomial logit model assigns a score $v(x)$ to every alternative x. The probability of selecting x from choice set C is

$$P(x \mid C) = \frac{\exp(v(x))}{\sum_{y \in C} \exp(v(y))}. \tag{3}$$

When the choice set contains two alternatives, this gives the Bradley–Terry model.

2.4 Bradley–Terry Model and Maximum Likelihood

2.4.1 Pairwise Comparison Setting

To understand the underlying mechanics of a model for choice data, we consider observed pairwise comparisons and assume a probabilistic model. The Bradley–Terry model, which is originally developed for analysing paired comparisons, provides a probabilistic framework for estimating the relative strengths of competing items [2].

The goal is to estimate unknown parameters that provide a compressed representation of the observed comparison data. For items i and j, with parameters θ_i and θ_j, the Bradley–Terry

model defines

$$P(i \succ j) = \frac{\exp(\theta_i)}{\exp(\theta_i) + \exp(\theta_j)}. \tag{4}$$

Dividing the numerator and denominator by $\exp(\theta_i)$ gives

$$P(i \succ j) = \frac{1}{1 + \exp\left(-(\theta_i - \theta_j)\right)}. \tag{5}$$

Therefore, the probability depends only on the parameter difference $\theta_i - \theta_j$.

2.4.2 Parameter Identifiability and Centring

The Bradley–Terry probability depends only on the difference between two parameters as see in equation (5). Consequently, adding the same constant c to every parameter does not change any pairwise comparison probability.

Therefore, the parameter vectors

$$(\theta_1, \ldots, \theta_n) \qquad \text{and} \qquad (\theta_1 + c, \ldots, \theta_n + c)$$

represent the same Bradley–Terry model. The individual parameter values are consequently not identifiable without an additional constraint.

We remove this additive ambiguity using the zero-sum constraint

$$\sum_{i=1}^{n} \theta_i = 0. \tag{6}$$

For any parameter vector $\boldsymbol{\theta}$, a centred version can be obtained by subtracting its mean:

$$\theta_i^{\text{centred}} = \theta_i - \frac{1}{n} \sum_{j=1}^{n} \theta_j. \tag{7}$$

This transformation preserves every parameter difference and therefore does not change the pairwise comparison probabilities. In the numerical experiments, both the true and estimated parameter vectors are centred before their recovery error is calculated.

Shift Invariance and the Zero-Sum Constraint The same additive ambiguity occurs in MNL: adding a common constant to all scores leaves the choice probabilities unchanged. Centring removes this ambiguity by selecting scores whose sum is zero.

Consider two alternatives, car and taxi, with scores $v(C) = 2.5$ and $v(T) = -0.9$. Their mean is 0.8. Subtracting this mean gives

$$v(C) = 2.5 - 0.8 = 1.7, \qquad v(T) = -0.9 - 0.8 = -1.7.$$

The centred scores sum to zero, while their difference remains 3.4. Therefore, the probability of choosing car is unchanged:

$$\frac{\exp(2.5)}{\exp(2.5) + \exp(-0.9)} = \frac{\exp(1.7)}{\exp(1.7) + \exp(-1.7)} \approx 0.968.$$

Thus, adding the same constant to every score does not change the choice probabilities. The zero-sum constraint selects one representative from all score vectors that produce the same probabilities.

2.4.3 Maximum Likelihood Estimation

The model parameters can be estimated from observed pairwise outcomes using maximum likelihood. Likelihood-based estimation and computational algorithms for extensions of the Bradley–Terry model have been studied in the literature [3].

Suppose that the observed dataset contains m pairwise comparisons:

$$\mathcal{D} = \{(w_t, l_t)\}_{t=1}^{m}, \tag{8}$$

where w_t and l_t denote the winner and loser of comparison t, respectively.

Conditional on the compared pairs, the observed comparison outcomes are modelled as independent. The probability of the observed outcome in comparison t is

$$p_t = P(w_t \succ l_t) = \frac{\exp(\theta_{w_t})}{\exp(\theta_{w_t}) + \exp(\theta_{l_t})}. \tag{9}$$

The likelihood of the complete dataset is therefore

$$L(\boldsymbol{\theta}; \mathcal{D}) = \prod_{t=1}^{m} \frac{\exp(\theta_{w_t})}{\exp(\theta_{w_t}) + \exp(\theta_{l_t})}. \tag{10}$$

Taking the logarithm gives the log-likelihood

$$\begin{aligned} \ell(\boldsymbol{\theta}; \mathcal{D}) &= \sum_{t=1}^{m} \log\left(\frac{\exp(\theta_{w_t})}{\exp(\theta_{w_t}) + \exp(\theta_{l_t})}\right) \\ &= \sum_{t=1}^{m} \left[\theta_{w_t} - \log\left(\exp(\theta_{w_t}) + \exp(\theta_{l_t})\right)\right]. \end{aligned}$$

Maximizing the log-likelihood is equivalent to minimizing its negative. In the implementation, we use the average negative log-likelihood

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m}\ell(\boldsymbol{\theta}; \mathcal{D}) \tag{11}$$

$$= \frac{1}{m}\sum_{t=1}^{m} \log\left(1 + \exp(\theta_{l_t} - \theta_{w_t})\right). \tag{12}$$

Using the average rather than the total negative log-likelihood makes the reported loss values comparable across datasets containing different numbers of comparisons.

The maximum likelihood estimate is written as

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}), \qquad \text{subject to} \qquad \sum_{i=1}^{n} \theta_i = 0. \tag{13}$$

2.4.4 Gradient-Based Estimation

The average negative log-likelihood in eq. (12) is minimized using gradient descent. For one observed winner–loser pair (w_t, l_t), the loss contribution is

$$\mathcal{L}_t = \log\left(1 + \exp(\theta_{l_t} - \theta_{w_t})\right). \tag{14}$$

Let

$$p_t = P(w_t \succ l_t). \tag{15}$$

The derivative of this loss with respect to the winner parameter is

$$\frac{\partial \mathcal{L}_t}{\partial \theta_{w_t}} = p_t - 1, \tag{16}$$

while the derivative with respect to the loser parameter is

$$\frac{\partial \mathcal{L}_t}{\partial \theta_{l_t}} = 1 - p_t. \tag{17}$$

Therefore, when the observed winner is assigned a small winning probability, the magnitude of the update is larger. When the model already assigns the observed winner a probability close to one, the magnitude of the update is smaller.

The gradient of the average negative log-likelihood with respect to an arbitrary parameter θ_k is

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \frac{1}{m}\sum_{t=1}^{m}\left[(p_t - 1)\mathbb{I}\{k = w_t\} + (1 - p_t)\mathbb{I}\{k = l_t\}\right], \tag{18}$$

where $\mathbb{I}\{\cdot\}$ is the indicator function.

At gradient-descent iteration s, the complete parameter vector is updated according to

$$\boldsymbol{\theta}^{(s+1)} = \boldsymbol{\theta}^{(s)} - \eta \nabla \mathcal{L}\left(\boldsymbol{\theta}^{(s)}\right), \tag{19}$$

where $\eta > 0$ is the learning rate.

After an update, the parameter vector is centred using

$$\theta_i^{(s+1)} \leftarrow \theta_i^{(s+1)} - \frac{1}{n}\sum_{j=1}^{n}\theta_j^{(s+1)}. \tag{20}$$

This centring step enforces the zero-sum constraint while preserving all pairwise parameter differences. The numerical experiments use this gradient-based procedure to estimate the Bradley–Terry parameters from the simulated winner–loser observations.

2.4.5 Existence and Uniqueness of the MLE

Ford's Condition For the Bradley–Terry model, the existence of a finite maximum likelihood estimate depends on the observed comparison outcomes. These outcomes can be represented using a directed win graph, in which each vertex represents an item and an edge $i \to j$ indicates

that item i has defeated item j.

Ford's condition holds when this directed win graph is strongly connected. Equivalently, for every nonempty proper subset of items, there must be at least one observed win from an item inside the subset against an item outside it, and vice versa.

When Ford's condition holds, the Bradley–Terry maximum likelihood estimate is finite. After imposing the centring constraint

$$\sum_{i=1}^{n} \theta_i = 0,$$

the estimate is unique.

2.5 From Multinomial Logit to Context-Dependent Choice Models

2.5.1 Universal Logit Representation

The universal logit model provides a framework for modelling choice probabilities [4]. In a universal logit model, the utility of an alternative can depend on the choice set in which it appears. Instead of assigning only one fixed utility to an alternative, the model allows it to have different utilities in different contexts. The probability of selecting x from C is

$$P(x \mid C) = \frac{\exp(u(x \mid C))}{\sum_{y \in C} \exp(u(y \mid C))}. \tag{21}$$

An identification constraint can be imposed separately for each choice set:

$$\sum_{y \in C} u(y \mid C) = 0, \qquad \text{for every } C \subseteq \mathcal{X}. \tag{22}$$

2.5.2 Decomposition of Context-Dependent Utility

Statement of Lemma 1

Lemma 2.1 (Decomposition of context-dependent utility)**.** *Related approaches also model the influence of competing alternatives on choice probabilities, including the market-share framework proposed by Batsell and Polking [1]. For a universal logit model, the context-dependent utility $u(x \mid C)$ can be written as a sum of terms indexed by subsets B of $C \setminus \{x\}$ [5]:*

$$u(x \mid C) = \sum_{B \subseteq C \setminus \{x\}} v(x \mid B). \tag{23}$$

Equivalently,

$$\begin{aligned} u(x \mid C) = v(x) &+ \sum_{y \in C \setminus \{x\}} v(x \mid \{y\}) \\ &+ \sum_{\{y,z\} \subseteq C \setminus \{x\}} v(x \mid \{y, z\}) + \cdots . \end{aligned} \tag{24}$$

The parameters satisfy

$$\sum_{x \notin B} v(x \mid B) = 0 \qquad \textit{for every } B \subseteq \mathcal{X}. \tag{25}$$

Interpretation of the Different Orders The term $v(x)$ represents the intrinsic preference associated with alternative x. It does not depend on the other alternatives present in the choice set. The term $v(x \mid \{z\})$ represents the contextual contribution of alternative z to the utility of alternative x. Similarly, a term such as $v(x \mid \{y, z\})$ represents a higher-order interaction involving the joint presence of alternatives y and z.

Worked Example Consider the choice set

$$A = \{C, T, Bi\}. \tag{26}$$

The subsets of A are

$$\emptyset,\ \{C\},\ \{T\},\ \{Bi\},\ \{C, T\},\ \{T, Bi\},\ \{Bi, C\},\ \{C, T, Bi\}. \tag{27}$$

The utility expansions are

$$\begin{aligned}
u(C \mid A) &= v(C) + v(C \mid \{T\}) + v(C \mid \{Bi\}) + v(C \mid \{T, Bi\}),\\
u(T \mid A) &= v(T) + v(T \mid \{C\}) + v(T \mid \{Bi\}) + v(T \mid \{C, Bi\}),\\
u(Bi \mid A) &= v(Bi) + v(Bi \mid \{C\}) + v(Bi \mid \{T\}) + v(Bi \mid \{C, T\}).
\end{aligned}$$

The corresponding constraints are

$$\begin{aligned}
v(C) + v(T) + v(Bi) &= 0,\\
v(T \mid \{C\}) + v(Bi \mid \{C\}) &= 0,\\
v(C \mid \{T\}) + v(Bi \mid \{T\}) &= 0,\\
v(C \mid \{Bi\}) + v(T \mid \{Bi\}) &= 0,\\
v(C \mid \{T, Bi\}) &= 0,\\
v(T \mid \{C, Bi\}) &= 0,\\
v(Bi \mid \{C, T\}) &= 0.
\end{aligned}$$

The final three constraints force the highest-order terms to zero. Therefore, the utilities reduce to

$$\begin{aligned}
u(C \mid A) &= v(C) + v(C \mid \{T\}) + v(C \mid \{Bi\}),\\
u(T \mid A) &= v(T) + v(T \mid \{C\}) + v(T \mid \{Bi\}),\\
u(Bi \mid A) &= v(Bi) + v(Bi \mid \{C\}) + v(Bi \mid \{T\}).
\end{aligned}$$

There are nine remaining parameters and four independent constraints. Hence, the number of free parameters is

$$9 - 4 = 5. \tag{28}$$

2.5.3 Hierarchy of Choice Models

The decomposition gives a hierarchy of choice models. A lower-order model retains simpler contextual interactions, while a higher-order model allows the utility of an alternative to depend on larger subsets of the choice set. Increasing the order makes the model more flexible, but also

increases the number of parameters that must be estimated.

The Model Classes $\mathcal{M}_p$ Let $\mathcal{M}_p$ denote the model obtained by retaining utility terms up to order p and setting all higher-order terms equal to zero. This produces the nested hierarchy

$$\mathcal{M}_1 \subseteq \mathcal{M}_2 \subseteq \cdots \subseteq \mathcal{M}_{n-1}. \tag{29}$$

The first-order model retains only $v(x)$. The second-order model additionally retains terms of the form $v(x \mid \{z\})$. Higher-order models include terms involving larger subsets of alternatives.

Number of Free Parameters In general, the number of free parameters in the order-p model is

$$d_p = \sum_{q=1}^{p} \binom{n}{q-1}(n-q), \qquad p = 1, \ldots, n-1. \tag{30}$$

For each context subset of size $q-1$, there are $n-q+1$ possible target alternatives. Their parameters satisfy one zero-sum constraint, leaving $n-q$ free parameters. There are $\binom{n}{q-1}$ such subsets [5].

For the first-order model (MNL) and the full universal logit model, this gives

$$d_1 = n - 1, \tag{31}$$

$$d_{n-1} = (n-2)2^{n-1} + 1. \tag{32}$$

For fixed p, the parameter count grows as $O(n^p)$: the first-order model has $O(n)$ parameters, the second-order model has $O(n^2)$, and higher orders require increasingly many parameters.

For the second-order model, there are n intrinsic parameters of the form $v(x)$ and $n(n-1)$ contextual parameters of the form $v(x \mid \{z\})$, where $x \neq z$. Thus, before applying the constraints, the number of parameters is

$$n + n(n-1) = n^2. \tag{33}$$

The intrinsic parameters satisfy one constraint:

$$\sum_{x \in \mathcal{X}} v(x) = 0. \tag{34}$$

For every $z \in \mathcal{X}$, the contextual parameters satisfy

$$\sum_{x \in \mathcal{X} \setminus \{z\}} v(x \mid \{z\}) = 0. \tag{35}$$

There are n contextual constraints. Therefore, the total number of constraints is $n+1$, and the number of free parameters is

$$n^2 - (n+1) = n^2 - n - 1. \tag{36}$$

2.5.4 Context-Dependent Random Utility Model

The first-order model does not allow the utility of an alternative to change when other alternatives are added to or removed from the choice set. To represent such context effects without retaining every higher-order interaction, we consider the second-order model $\mathcal{M}_2$.

Second-Order Truncation Starting from the universal logit decomposition,

$$\begin{aligned} u(x \mid C) = v(x) &+ \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}) \\ &+ \sum_{\{y,z\} \subseteq C \setminus \{x\}} v(x \mid \{y,z\}) + \cdots, \end{aligned} \tag{37}$$

the second-order model sets all terms involving two or more contextual alternatives equal to zero. Hence,

$$u(x \mid C) = v(x) + \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}). \tag{38}$$

The corresponding choice probability is

$$P(x \mid C) = \frac{\exp\left(v(x) + \sum_{z \in C \setminus \{x\}} v(x \mid \{z\})\right)}{\sum_{y \in C} \exp\left(v(y) + \sum_{z \in C \setminus \{y\}} v(y \mid \{z\})\right)}. \tag{39}$$

Here, $v(x)$ represents the intrinsic preference associated with x, while $v(x \mid \{z\})$ represents the contribution of context alternative z to the utility of x.

Reparameterisation of the Model The second-order utility is given by

$$u(x \mid C) = v(x) + \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}). \tag{40}$$

For $x \neq z$, define the pairwise interaction parameter as

$$u_{xz} = v(x \mid \{z\}) - v(z). \tag{41}$$

Equivalently,

$$v(x \mid \{z\}) = u_{xz} + v(z).$$

Substituting this into the original utility,

$$u(x \mid C) = v(x) + \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}) \tag{42}$$

$$= v(x) + \sum_{z \in C \setminus \{x\}} u_{xz} + \sum_{z \in C \setminus \{x\}} v(z) \tag{43}$$

$$= \underbrace{v(x) + \sum_{z \in C \setminus \{x\}} v(z)}_{\sum_{z \in C} v(z)} + \sum_{z \in C \setminus \{x\}} u_{xz} \tag{44}$$

$$= \sum_{z \in C} v(z) + \sum_{z \in C \setminus \{x\}} u_{xz}. \tag{45}$$

The first term depends only on the choice set C and is the same for every alternative $x \in C$. Since adding or subtracting a common constant does not change the softmax probabilities, this term can be omitted from the utility representation.

Thus, the reparameterised utility becomes

$$\boxed{u(x \mid C) = \sum_{z \in C \setminus \{x\}} u_{xz}.} \tag{46}$$

Using the reparameterised utility, the choice probability of alternative x from choice set C is given by

$$P(x \mid C) = \frac{\exp(u(x \mid C))}{\sum_{y \in C} \exp(u(y \mid C))} \tag{47}$$

$$= \frac{\exp\left(\sum_{z \in C \setminus \{x\}} u_{xz}\right)}{\sum_{y \in C} \exp\left(\sum_{z \in C \setminus \{y\}} u_{yz}\right)}. \tag{48}$$

Thus, the Context-Dependent Model (CDM) is expressed as

$$\boxed{P(x \mid C; U) = \frac{\exp\left(\sum_{z \in C \setminus \{x\}} u_{xz}\right)}{\sum_{y \in C} \exp\left(\sum_{z \in C \setminus \{y\}} u_{yz}\right)}}. \tag{49}$$

Number of Parameters The reparameterised parameters can be arranged in a matrix $U \in \mathbb{R}^{n \times n}$. The diagonal entries are set to zero because an alternative is not treated as its own context:

$$u_{xx} = 0. \tag{50}$$

Therefore, U contains

$$n^2 - n = n(n-1) \tag{51}$$

off-diagonal parameters. These parameters satisfy the identification constraint

$$\sum_{x \in \mathcal{X}} \sum_{z \in \mathcal{X} \setminus \{x\}} u_{xz} = 0. \tag{52}$$

Hence, the number of free parameters is

$$n(n-1) - 1 = n^2 - n - 1. \tag{53}$$

This agrees with the count obtained from the original second-order formulation in eq. (36). For example, suppose $\mathcal{X} = \{A, B, C\}$. Then

$$U = \begin{pmatrix} 0 & u_{AB} & u_{AC} \\ u_{BA} & 0 & u_{BC} \\ u_{CA} & u_{CB} & 0 \end{pmatrix}. \tag{54}$$

The matrix has six off-diagonal parameters. After applying the constraint

$$u_{AB} + u_{AC} + u_{BA} + u_{BC} + u_{CA} + u_{CB} = 0, \tag{55}$$

the number of free parameters is

$$6 - 1 = 5. \tag{56}$$

This is consistent with the five free parameters obtained in eq. (28).

2.5.5 Low-Rank Context-Dependent Model

In the full context-dependent model, the parameter u_{xz} represents the effect of context alternative z on target alternative x. There are $n(n-1)$ off-diagonal interaction parameters for n alternatives. The low-rank model instead constructs these interactions from smaller target and context vectors.

Each alternative x has a target vector t_x and a context vector c_x, each with r components. For two distinct alternatives, their interaction is

$$u_{xz} = t_x^\top c_z, \qquad x \neq z. \tag{57}$$

The value r is selected before fitting the model. The target and context vectors are parameters learned from choice data; they are not observed features supplied in the dataset.

If the target vectors form the rows of T and the context vectors form the rows of C, their matrix product is

$$M = TC^\top, \qquad T, C \in \mathbb{R}^{n \times r}. \tag{58}$$

Rows of M correspond to targets and columns correspond to contexts. Its off-diagonal entries are the CDM interactions. An alternative is not its own context, so diagonal entries are ignored in computing choice probabilities.

The factor product M has rank at most r. Setting its diagonal to zero may change the matrix rank; therefore, the rank restriction refers to the factor product $TC^\top$.

3 Experiments and Results

3.1 Bradley–Terry Experiments

3.1.1 Bradley–Terry Recovery versus Number of Items Using Choix

Source notebook: `00_BTL_recovery_noofitems.ipynb`

This notebook studies how accurately the true parameters can be recovered from simulated comparison data. We use $n \in \{10, 20, 50, 100, 200\}$ and repeat each setting five times. The true parameters are sampled from a standard normal distribution and centred. Pairs are sampled uniformly, and parameters are fitted using the Choix library.

Number of Items at Fixed Total Comparisons The total number of comparisons is fixed at $m \in \{1000, 5000, 20000, 100000\}$ while the number of items increases. For a fixed number of

comparisons, increasing the number of items reduces the expected number of appearances per item, $2m/n$.

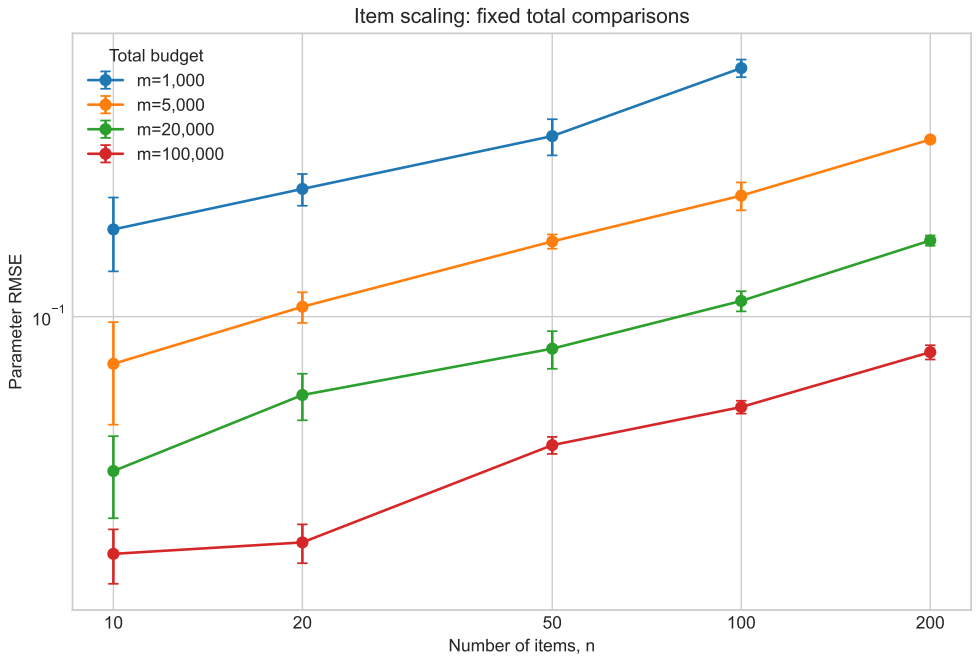


Figure 2: Choix parameter-recovery RMSE versus the number of items at fixed total comparison budgets. Points show means over converged runs, and error bars show one standard deviation.

Number of Items at Fixed Comparisons per Item We also compare settings with $m/n = 100$ and $m/n = 500$. Here, the total number of comparisons increases with the number of items, so the expected appearances per item remain fixed at 200 and 1000, respectively.

Only runs with a finite MLE and a converged fit are included in the recovery summaries. At $n = 200$ and $m = 1000$, none of the five runs has a finite MLE, so no RMSE point is shown for that setting.

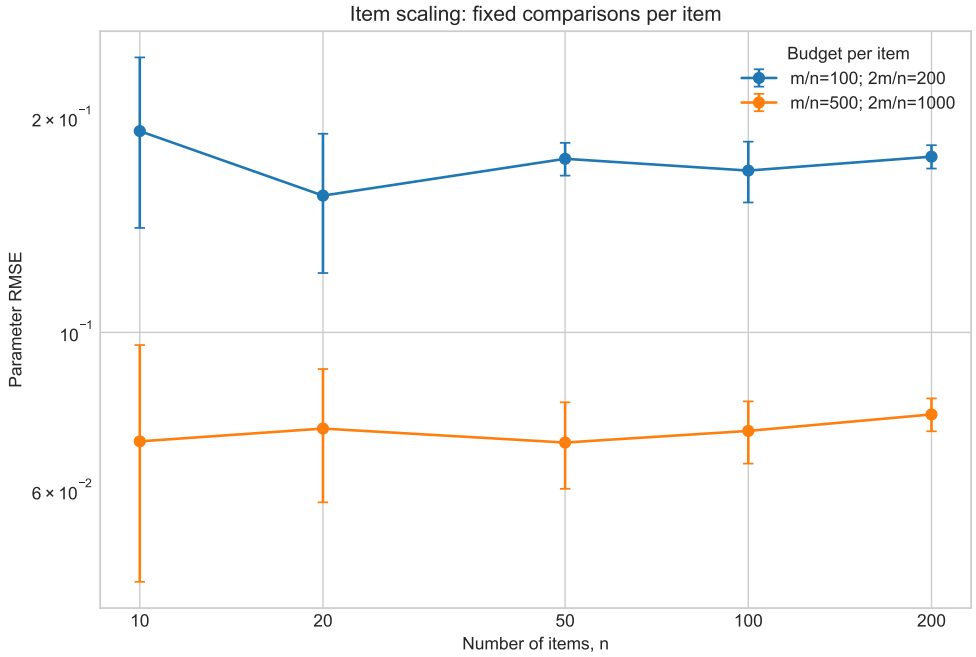


Figure 3: Choix parameter-recovery RMSE versus the number of items at fixed m/n. Error bars show one standard deviation across converged runs.

3.1.2 Parameter Recovery Using Gradient Descent

Source notebook: `01_BradleyTerry_recovery.ipynb`

Baseline Parameter Recovery The first experiment checks whether the parameters used to generate simulated Bradley–Terry comparisons can be recovered from the observed data.

We consider $n = 10$ items. The true parameter vector is constructed using equally spaced values between -1.5 and 1.5. The vector is centred so that

$$\sum_{i=1}^{n} \theta_i = 0. \tag{59}$$

A total of $m = 1000$ pairwise comparisons are generated. Each pair is sampled uniformly, and the winner is sampled according to the Bradley–Terry probability in eq. (4). The random seed is fixed at 20260831 to make the experiment reproducible.

The unknown parameters are estimated by minimizing the average negative log-likelihood using gradient descent. The learning rate is set to 0.1, and the optimizer is run for a maximum of 1000 iterations.

The parameter-recovery error is measured using the root mean squared error

$$\mathrm{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \left(\widehat{\theta}_i - \theta_i \right)^2}, \tag{60}$$

where θ_i is the true parameter and $\widehat{\theta}_i$ is its estimate.

As shown in fig. 4, the estimated parameters follow the ordering and approximate magnitudes of

Table 1: Configuration and result of the baseline Bradley–Terry parameter-recovery experiment.

Quantity	Value
Number of items, n	10
Number of comparisons, m	1000
Random seed	20260831
Learning rate	0.1
Maximum iterations	1000
Parameter-recovery RMSE	0.1224

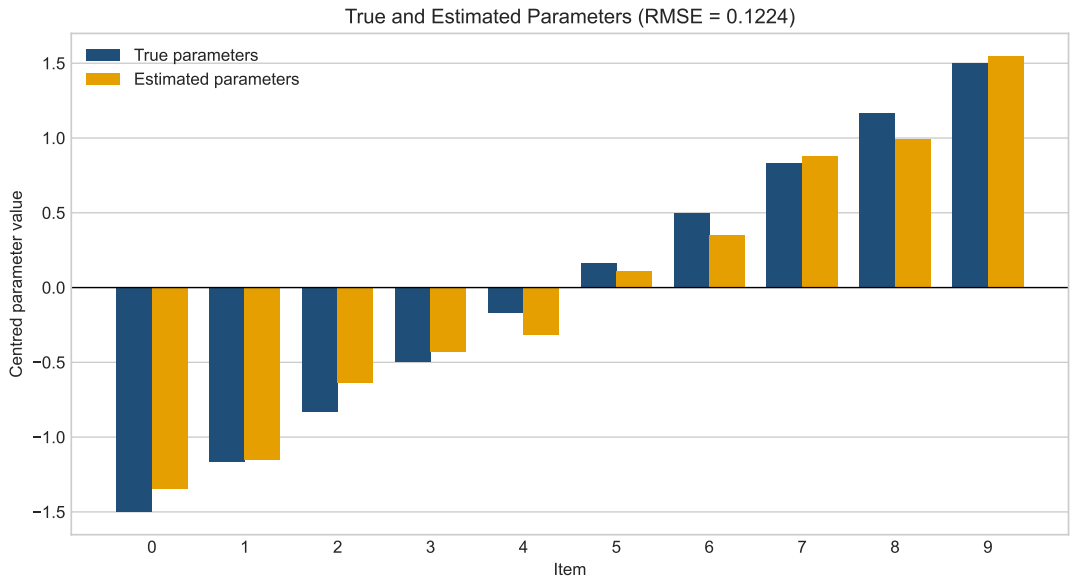


Figure 4: Comparison of the true and estimated parameters for the baseline Bradley–Terry experiment with 10 items and 1000 pairwise comparisons.

the true parameters. The parameter-recovery RMSE for this simulated dataset is 0.1224.

This result provides a baseline example of parameter recovery. Since it is obtained from a single simulated dataset, it does not describe the variation across independent datasets. Repeated experiments are used in the next subsection to study how the recovery error changes with the number of observed comparisons.

Effect of the Number of Comparisons The second experiment studies how the number of observed pairwise comparisons affects parameter recovery. The number of items is fixed at $n = 10$, and the same centred true parameter vector used in the baseline experiment is retained.

The number of comparisons is varied over

$$m \in \{50, 100, 200, 500, 1000, 2000, 5000\}. \tag{61}$$

For each value of m, the experiment is repeated independently 20 times. A new comparison dataset is generated in every run, while the true parameters, pair-sampling method, learning rate, and maximum number of gradient-descent iterations are kept fixed. The mean parameter-recovery RMSE and its standard deviation are then computed over the 20 runs.

Table 2: Parameter-recovery error for different numbers of pairwise comparisons. Each result is computed over 20 independent runs.

Number of comparisons	Mean RMSE	Standard deviation
50	0.7225	0.2172
100	0.4600	0.1247
200	0.3213	0.0819
500	0.1786	0.0487
1000	0.1412	0.0388
2000	0.1095	0.0309
5000	0.0780	0.0203

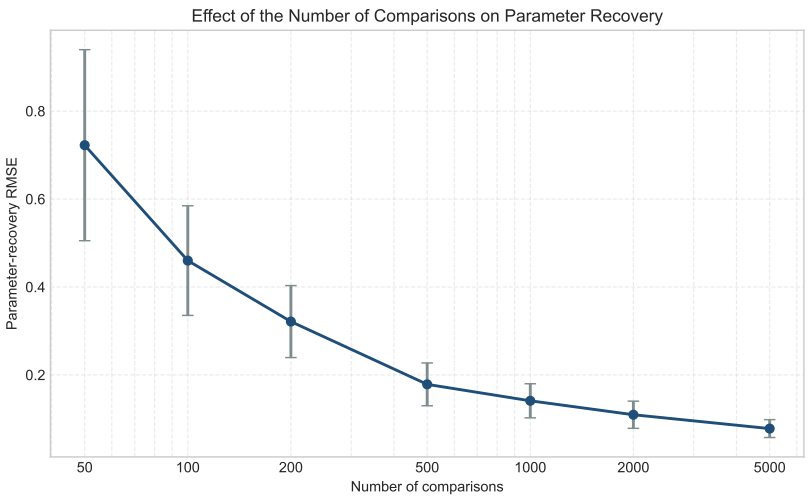


Figure 5: Mean Bradley–Terry parameter-recovery RMSE as the number of comparisons increases. The error bars show one standard deviation across 20 independent runs.

As shown in table 2 and fig. 5, the mean parameter-recovery RMSE decreases as the number of

observed comparisons increases. It decreases from 0.7225 with 50 comparisons to 0.0780 with 5000 comparisons.

The standard deviation also decreases from 0.2172 to 0.0203. Thus, in this experimental setting, increasing the amount of observed comparison data improves both the average recovery accuracy and the consistency of the estimates across independent datasets.

3.1.3 Comparison of Optimization Methods

Source notebook: `04_BradleyTerry_MLE_Optimizations.ipynb`

Experimental Setup The Bradley–Terry negative log-likelihood was minimized using five optimization methods: full-batch gradient descent, stochastic gradient descent (SGD), mini-batch gradient descent, gradient descent with momentum, and Adam.

Each method was implemented in two ways: a handcrafted implementation and an implementation using PyTorch. The same simulated comparison dataset and initial parameter vector were used for all methods.

A maximum budget of 10,000 epochs was used. Convergence was measured using the norm of the gradient evaluated on the complete dataset. The convergence tolerance was set to

$$\|\nabla \mathcal{L}(\boldsymbol{\theta})\|_2 \leq 10^{-4}. \tag{62}$$

For easier comparison, define the convergence ratio

$$r = \frac{\|\nabla \mathcal{L}(\boldsymbol{\theta})\|_2}{10^{-4}}. \tag{63}$$

Thus, a run satisfies the convergence criterion when $r \leq 1$.

Final Solution Quality Table 3 summarizes the parameter-recovery error, convergence ratio, and relative runtime. Here, H and P denote the handcrafted and PyTorch implementations, respectively.

Table 3: Comparison of the optimization methods. The runtime ratio is PyTorch runtime divided by handcrafted runtime.

Optimizer	RMSE (H/P)	r (H/P)	Runtime ratio (P/H)
Full-batch GD	0.0381/0.0381	1.00/1.00	1.34
SGD	0.0393/0.0395	4.80/4.54	2.59
Mini-batch GD	0.0384/0.0387	0.90/6.31	27.82
Momentum GD	0.0404/0.0360	7.92/9.82	2.15
Adam	0.0408/0.0438	24.94/29.12	2.31

The recovery RMSE values are similar across the different methods, lying between 0.0360 and 0.0438. Therefore, all methods produced parameter estimates close to the true parameters in this experiment. However, similar recovery errors do not imply that every method satisfied the numerical convergence criterion.

Convergence Criterion Figure 6 shows the final gradient norm relative to the selected tolerance.

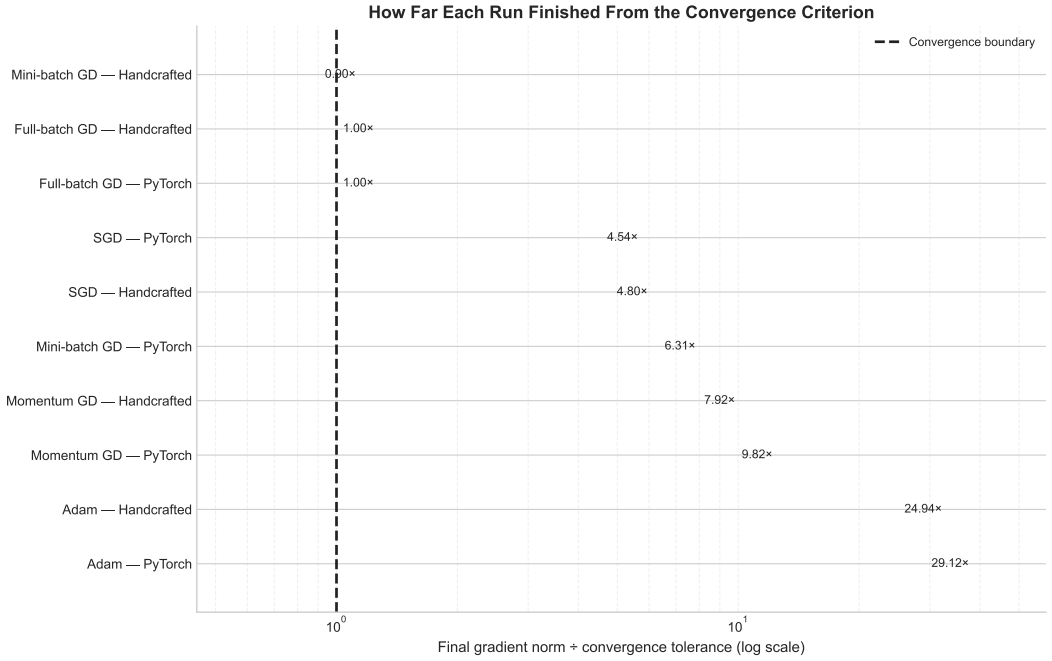


Figure 6: Final gradient norm relative to the convergence tolerance. Values at or below one satisfy the convergence criterion.

Both implementations of full-batch gradient descent satisfied the convergence criterion. The handcrafted mini-batch implementation also satisfied the criterion. The remaining methods did not reach the specified tolerance within the fixed epoch budget.

In particular, stochastic methods continue to fluctuate near a solution because their updates are computed using individual observations or mini-batches. Consequently, a small recovery error may still be obtained even when the full-gradient norm remains above the strict convergence tolerance.

Handcrafted and PyTorch Runtime Figure 7 compares the runtime of the PyTorch implementation with its corresponding handcrafted implementation.

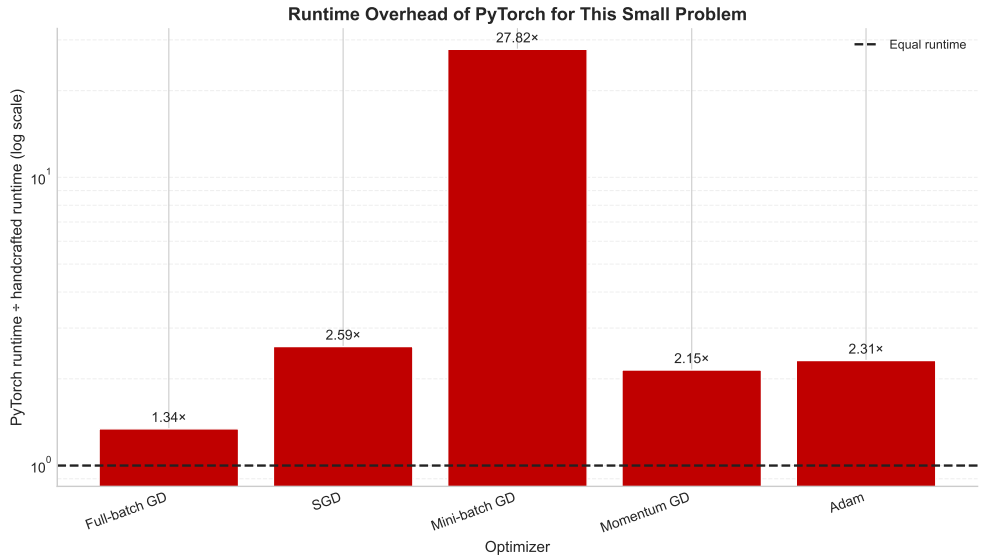


Figure 7: Ratio of PyTorch runtime to handcrafted runtime for each optimization method. A value greater than one indicates that the PyTorch implementation required more time.

For this Bradley–Terry problem, the handcrafted implementations were faster than their PyTorch counterparts, and among all those Adam optimizer performed well. The additional runtime is consistent with the overhead introduced by tensor operations and automatic differentiation on a small CPU-based problem.

This observation is specific to the present implementation and problem size. It should not be interpreted as a general comparison of PyTorch and handcrafted optimization, since PyTorch may become advantageous for larger models, larger datasets, or GPU-based computation.

3.2 Computational Scaling of the Full and Low-Rank CDM

Source notebook: `01-FullvsLowrankResourceScaling.ipynb`

The low-rank model uses fewer trainable parameters than the full CDM. We now examine how this reduction affects the resources required for model fitting as the number of items increases. The experiment compares training time, average time per update, parameter storage, and process memory. It uses the existing dense-matrix implementations of the models.

3.2.1 Experimental Setup

The number of items was varied over $n \in \{250, 500, 1000, 2000, 3000, 5000\}$. For each item count, we generated 1,000 observations from a full CDM with parameter scale 0.4 and choice-set sizes two and three. The full model and low-rank models with $r \in \{1, 2, 3\}$ were fitted to the same observations within each repeat. Five repeats were performed for every setting, giving 120 successful model fits in total. Dataset fingerprints were checked to verify that the four models received the same observations within each repeat.

All models were fitted using Adam with learning rate 0.05 for exactly 1,000 updates. Computation was performed on the CPU using `float64` and one PyTorch computational thread. The base random seed was 20260831. Each fit ran in a separate process, and the order of the fits was

randomly shuffled.

The training time includes the optimization loop. Data generation, model construction, and the final loss and gradient evaluation are outside this timing interval. Since the update budget is fixed, this experiment measures computational cost rather than time to convergence. It also does not compare parameter recovery or held-out predictive performance.

3.2.2 Number of Trainable Parameters and Parameter Storage

The full implementation stores an $n \times n$ trainable matrix. The low-rank implementation stores two $n \times r$ factor matrices. Thus, the numbers of trainable entries are

$$N_{\text{full}} = n^2, \qquad N_{\text{low-rank}} = 2nr. \tag{64}$$

These counts describe the entries stored by the implementation. They are not the numbers of identifiable free parameters discussed earlier. For a fixed rank, the number of trainable entries grows linearly with n, whereas the full matrix requires quadratic storage.

For example, at $n = 5000$, the full model contains 25,000,000 trainable entries. The rank-one, rank-two, and rank-three models contain 10,000, 20,000, and 30,000 entries, respectively. The rank-one model therefore uses 2,500 times fewer trainable entries than the full model.

Each `float64` entry occupies eight bytes. The storage of the trainable tensors, measured in MiB, is therefore

$$S_{\text{parameters}} = \frac{8N_{\text{parameters}}}{2^{20}}. \tag{65}$$

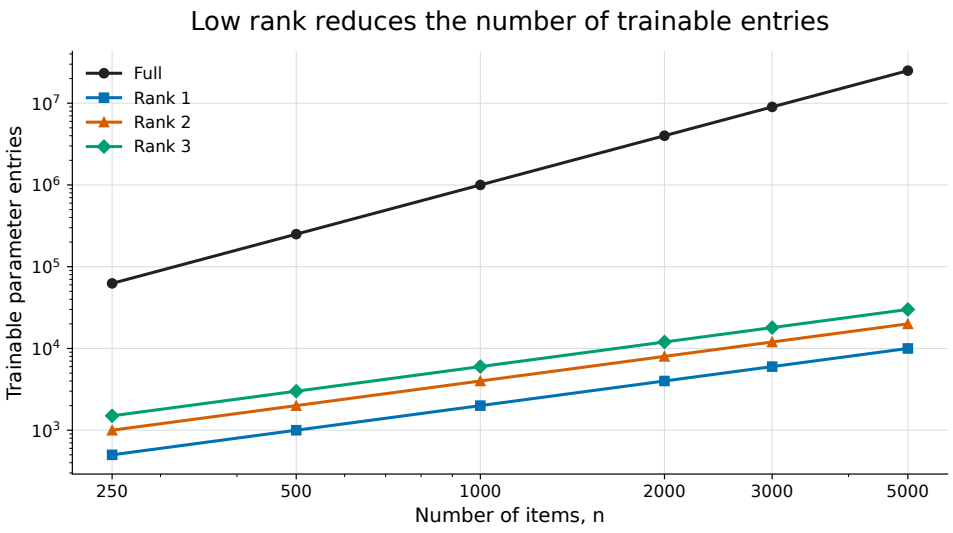
This quantity excludes gradients, optimizer states, observations, and temporary allocations. It should therefore be distinguished from the total memory occupied by the process.

3.2.3 Training Time and Average Time per Update

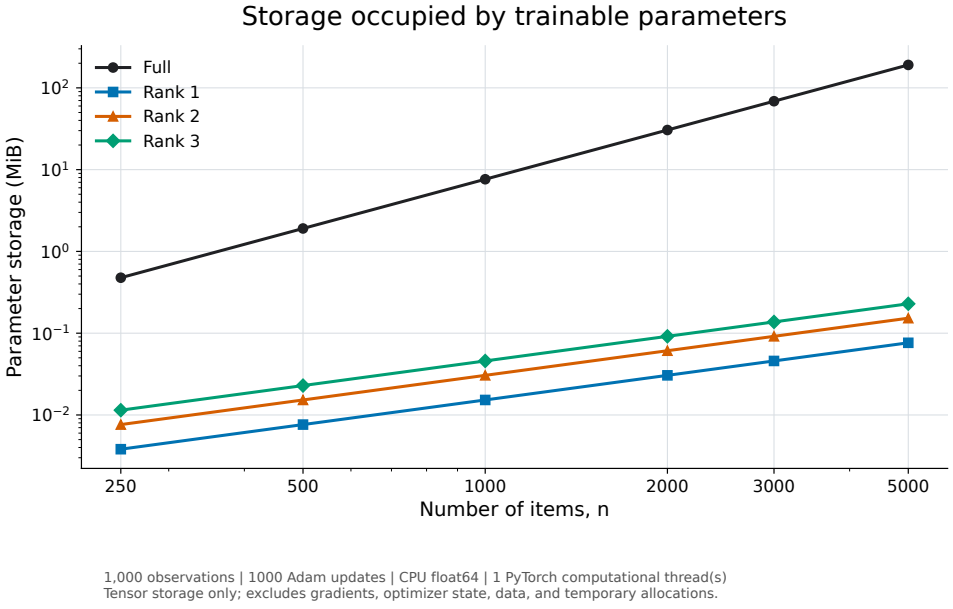
Table 4 reports the mean training time, its standard deviation, and the mean sampled peak process memory across the five repeats. Training time increased substantially with the number of items for all four models.

At $n = 5000$, the full model required 413.346 ± 2.486 seconds. The rank-one, rank-two, and rank-three models required 307.714 ± 4.728, 322.378 ± 4.124, and 319.222 ± 7.053 seconds, respectively. Here, the values after $\pm$ are standard deviations across the five repeats. The low-rank models were approximately 22–26% faster at this item count. This reduction is much smaller than the reduction in the number of trainable entries.

The average time per update was calculated by dividing the training-loop time by 1,000. It follows the same scaling pattern as total training time because every model uses the same update budget. Its error bars describe variation across repeats, rather than variation between individual updates within a fit.



(a) Number of trainable entries.

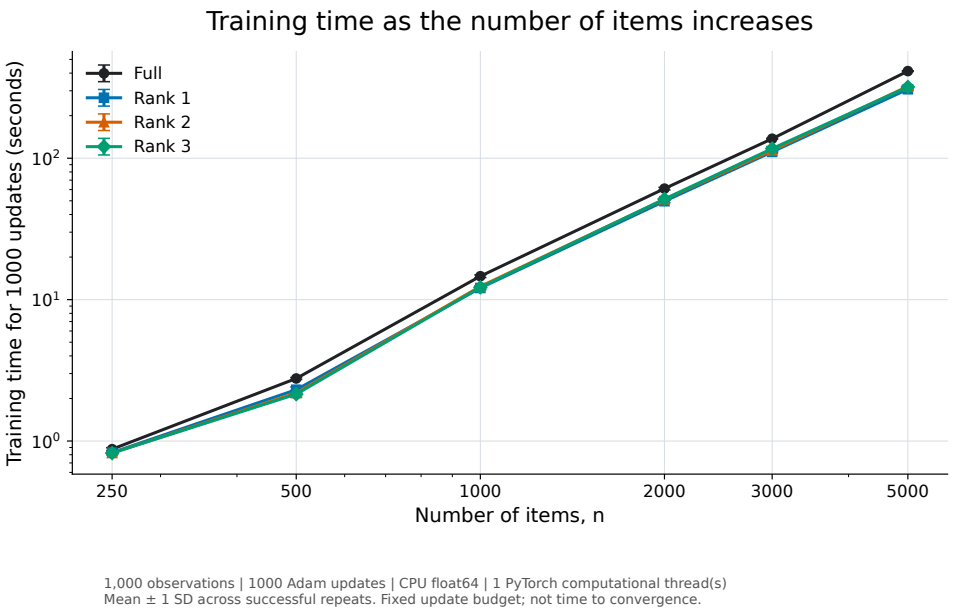


(b) Storage occupied by trainable tensors.

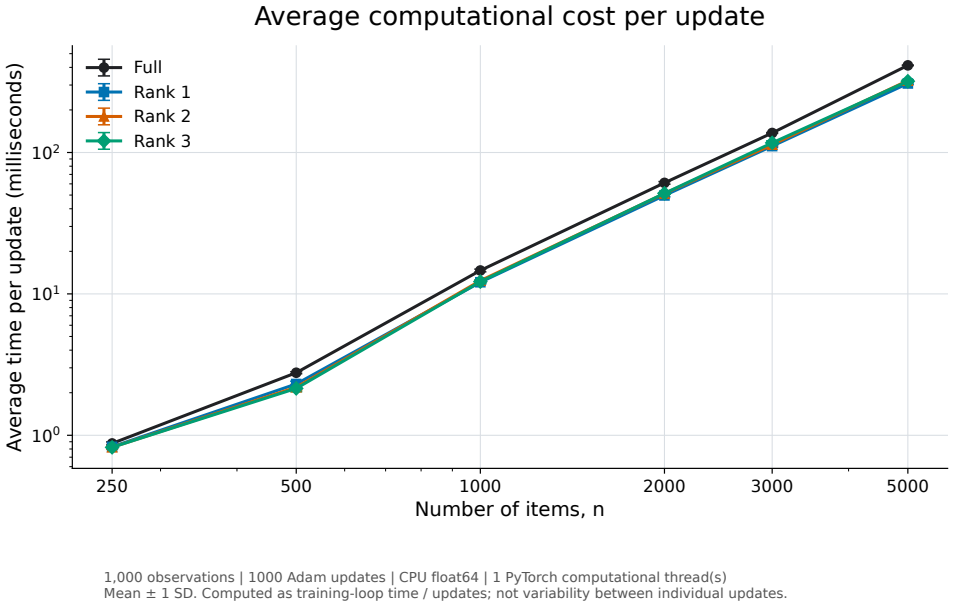
Figure 8: Trainable entries and their tensor storage for the full and low-rank CDM. Storage excludes gradients, optimizer states, data, and temporary allocations.

Table 4: Computational resources for the full and low-rank CDM. Time is measured over 1,000 Adam updates. Each setting has five repeats. Peak RSS is the mean sampled peak process memory.

Model	Items	Runs	Parameters	Time mean (s)	Time SD (s)	Peak RSS (MiB)
Full	250	5	62500	0.876	0.022	299.559
Full	500	5	250000	2.770	0.046	320.503
Full	1000	5	1000000	14.643	0.320	357.125
Full	2000	5	4000000	60.975	1.281	470.828
Full	3000	5	9000000	137.423	1.017	674.728
Full	5000	5	25000000	413.346	2.486	1249.312
Rank 1	250	5	500	0.829	0.020	312.606
Rank 1	500	5	1000	2.306	0.105	316.431
Rank 1	1000	5	2000	12.089	0.337	320.850
Rank 1	2000	5	4000	49.681	1.108	395.331
Rank 1	3000	5	6000	110.941	2.039	530.866
Rank 1	5000	5	10000	307.714	4.728	1023.025
Rank 2	250	5	1000	0.822	0.025	301.675
Rank 2	500	5	2000	2.186	0.101	309.366
Rank 2	1000	5	4000	12.400	0.137	343.231
Rank 2	2000	5	8000	51.217	0.622	418.922
Rank 2	3000	5	12000	113.587	3.033	528.475
Rank 2	5000	5	20000	322.378	4.124	1013.959
Rank 3	250	5	1500	0.822	0.010	287.931
Rank 3	500	5	3000	2.145	0.112	312.275
Rank 3	1000	5	6000	12.211	0.328	339.137
Rank 3	2000	5	12000	51.435	0.288	422.572
Rank 3	3000	5	18000	116.987	4.034	542.128
Rank 3	5000	5	30000	319.222	7.053	1018.731



(a) Total training-loop time.



(b) Average time per update.

Figure 9: Computational cost as the number of items increases. Points show means across five repeats, and error bars show one standard deviation. Each fit uses 1,000 Adam updates.

3.2.4 Empirical Growth of Training Time

To describe the observed runtime growth, we fitted a power law to the mean training times:

$$t(n) \approx an^{\alpha}, \qquad \log t(n) \approx \log a + \alpha \log n. \tag{66}$$

The intercept and exponent were estimated by ordinary least squares in log-log coordinates. Only the four item counts with $n \geq 1000$ were used, and each included setting had all five repeats completed.

Table 5: Empirical power-law fits to mean training time for $n \in \{1000, 2000, 3000, 5000\}$. The doubling multiplier is 2^{α}. Fit statistics are reported at the precision saved by the notebook.

Model	Exponent alpha	Log-log R squared	Time multiplier when n doubles	Item counts fitted
Full	2.069	1.000	4.197	4
Rank 1	2.010	1.000	4.029	4
Rank 2	2.021	1.000	4.059	4
Rank 3	2.030	1.000	4.083	4

The estimated exponents were 2.069 for the full model and 2.010, 2.021, and 2.030 for ranks one, two, and three, respectively. Thus, the measured training time grew approximately quadratically over the fitted range for all four models. Under these fitted relationships, doubling the number of items multiplies the training time by approximately 4.197, 4.029, 4.059, and 4.083, respectively.

These exponents describe the observed implementation and item range. They do not prove an asymptotic complexity bound. In particular, the linear growth of the number of low-rank trainable entries does not imply linear training time for the dense implementation used here.

3.2.5 Process Memory

Process memory was measured using resident set size (RSS), sampled at intervals of 0.005 seconds. The baseline was recorded after data generation and before model construction. Sampling continued through model fitting and the final loss and gradient evaluation. The sampled peak therefore includes the Python/PyTorch process, observations, model parameters, and allocations made during fitting. It is not a measurement of parameter storage alone.

At $n = 5000$, the mean sampled peak RSS was 1249.312 MiB for the full model. The corresponding values were 1023.025, 1013.959, and 1018.731 MiB for ranks one, two, and three, respectively. Low-rank parameterization reduced the measured peak at this item count, but the reduction was much smaller than the parameter-storage reduction. At the smaller item counts, process overhead and variation between runs also affect the ordering of the measured peaks.

The notebook additionally records the increase above the baseline:

$$\Delta\mathrm{RSS} = \mathrm{RSS}_{\text{sampled peak}} - \mathrm{RSS}_{\text{baseline}}. \tag{67}$$

Both memory measurements are approximate. Periodic sampling can miss short-lived peaks, and memory retained or reused by the allocator can affect the increase above the baseline.

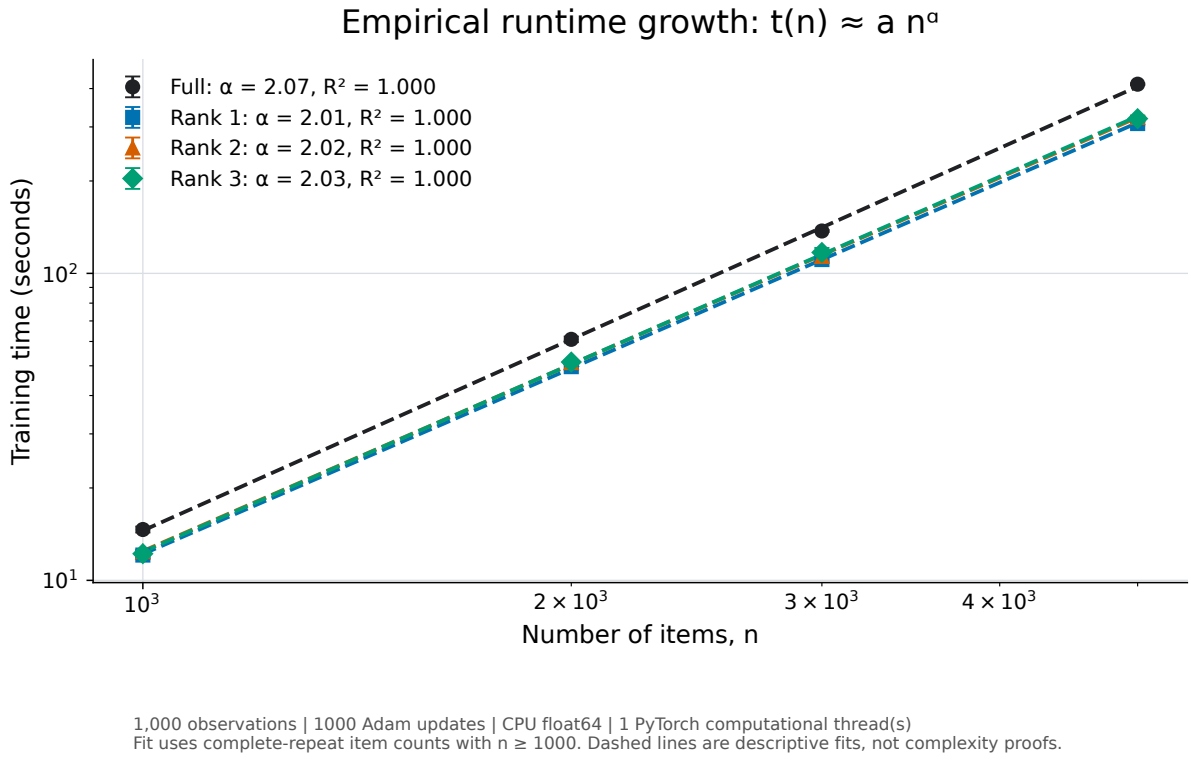


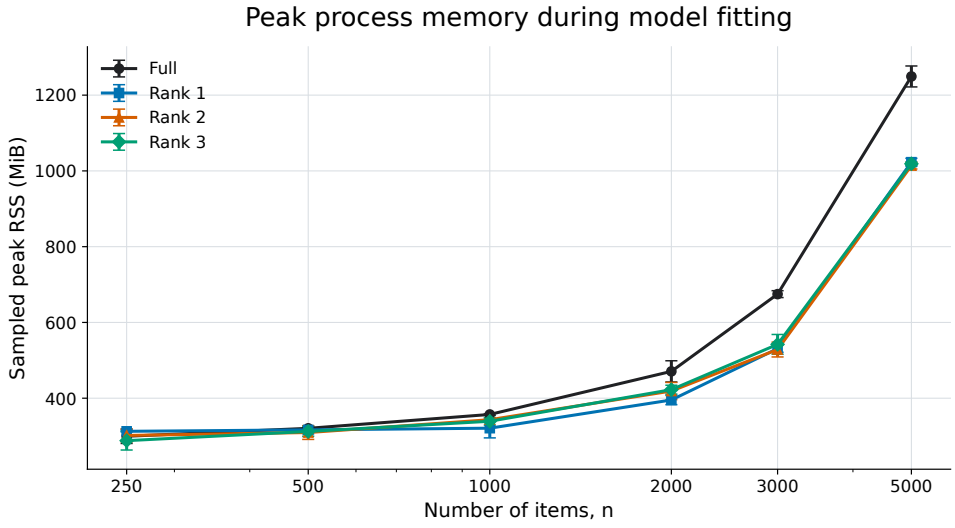
Figure 10: Empirical power-law fits to training time for $n \geq 1000$. Points and error bars show the mean and standard deviation across five repeats. Dashed lines describe the fitted range and are not proofs of asymptotic complexity.

3.2.6 Summary of scaling experiments.

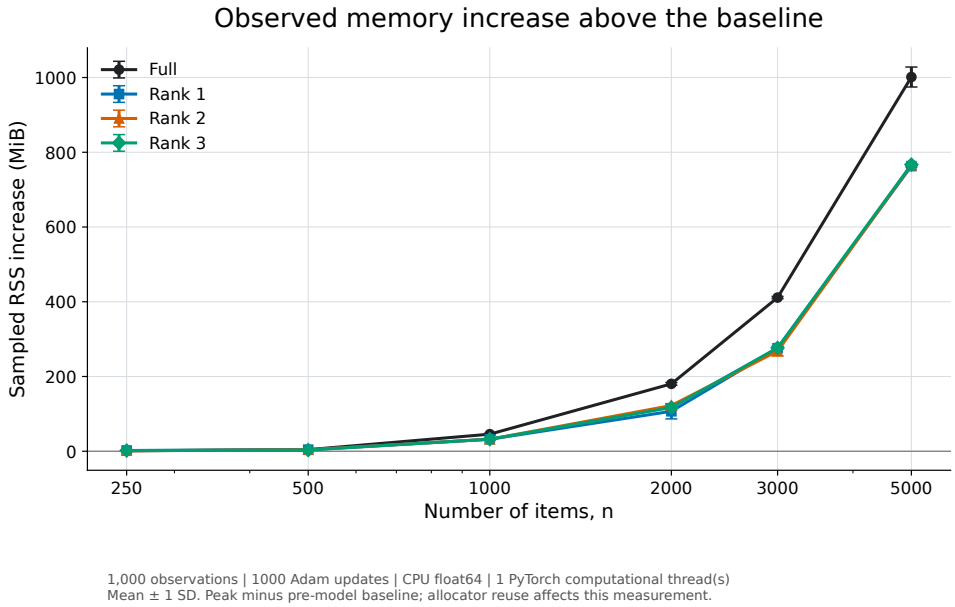
So, these experiments show that low-rank parameterization is substantially reducing the number of trainable parameters. However, parameter count, training time, and total process memory do not decrease by the same factor. In these present implementations, the low-rank models still show approximately quadratic runtime growth over the measured range. The results therefore separate the storage benefit of the parameterization from the computational behaviour of the implementation.

4 CDM Paper Reproduction

5 Discussion and Further Work



(a) Sampled peak RSS.



(b) Sampled peak RSS minus the pre-model baseline.

Figure 11: Sampled process memory during fitting. Points show means across five repeats, and error bars show one standard deviation. RSS includes process overhead, and periodic sampling can miss brief peaks.

References

[1] Richard R. Batsell and John C. Polking. A new class of market share models. *Marketing Science*, 4(3):177–198, 1985. doi: 10.1287/mksc.4.3.177.

[2] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: The method of paired comparisons. *Biometrika*, 39(3–4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[3] David R. Hunter. Mm algorithms for generalized bradley-terry models. *The Annals of Statistics*, 32(1):384–406, 2004. doi: 10.1214/aos/1079120141.

[4] Daniel McFadden. On the interpretation of the Logit model. *Working Paper*, 1975.

[5] Arjun Seshadri, Alexander Peysakhovich, and Johan Ugander. Discovering context effects from raw choice data. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5660–5669. PMLR, 2019.